The Journal of Systems and Software 83 (2010) 1004–1014

Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Scheduling multiple task graphs with end-to-end deadlines in distributed real-time systems utilizing imprecise computations

Georgios L. Stavrinides *, Helen D. Karatza

Department of Informatics, Aristotle University of Thessaloniki, 54124 Thessaloniki, Greece

ARTICLE INFO

Article history:
Received 2 October 2009
Received in revised form 13 December 2009
Accepted 21 December 2009
Available online 4 January 2010

Keywords:
Distributed real-time systems
Imprecise computations
Input error
Performance evaluation
Scheduling
Task graph

ABSTRACT

In order to meet the inherent need of real-time applications for high quality results within strict timing constraints, the employment of effective scheduling techniques is crucial in distributed real-time systems. In this paper, we evaluate by simulation the performance of strategies for the dynamic scheduling of composite jobs in a homogeneous distributed real-time system. Each job that arrives in the system is a directed acyclic graph of component tasks and has an end-to-end deadline. For each scheduling policy, we provide an alternative version which allows imprecise computations, taking into account the effects of input error on the processing time of the component tasks of a job. The simulation results show that the alternative versions of the algorithms outperform their respective counterparts. To our knowledge, an imprecise computations approach for the dynamic scheduling of multiple task graphs with end-to-end deadlines and input error has never been discussed in the literature before.

© 2009 Elsevier Inc. All rights reserved.

1. Introduction

Distributed real-time systems have become increasingly important in many domains of our daily life. Such systems are used for the control of nuclear power plants, financial markets, radar systems, telecommunication networks, medical care monitoring and multimedia applications. In a real-time system, the jobs have *deadlines* that must be met. The correctness of the system does not depend only on the logical results of the computations, but also on the time at which the results are produced. If a real-time job fails to meet its deadline, then its results will be useless, or even worse, this may have disastrous consequences for the system and the environment that is under control (Buttazzo, 2004).

Consequently, in order to guarantee that every real-time job will produce high quality results within the imposed timing constraints, the employment of effective scheduling techniques is crucial in distributed real-time systems. The scheduling algorithm is responsible for the allocation of processors to jobs and determines the order in which jobs will be executed on processors.

Based on the observation that in real-time systems it is often more desirable for a job to produce an approximate result by its deadline, than to produce an exact result late, Lin et al. proposed the *imprecise computations* technique (Lin et al., 1987). According to this technique, the execution of a real-time job is allowed to return intermediate (*imprecise*) results of poorer, but still acceptable quality, when the deadline of the job cannot be met. To achieve this, it is assumed that every job is *monotone*. That is, the accuracy of its intermediate results is increased as more time is spent to produce them. If the execution of a monotone job is fully completed, then the results are precise (Liu et al., 1991).

A video-on-demand server, for example, which streams movies and other video content to users over the internet, may unexpectedly encounter network congestion, causing some packets to be lost or to be corrupted during the transmission of a video to the user. In this case, imprecise computations can allow the system to reduce the quality of some video frames during the transmission, so that the video delivered to the user maintains an acceptable frame rate. Another example, is the collision detection system in aircraft. This system collects data from the airplane's radar, identifies possible collisions with nearby aircraft and notifies the pilot. Imprecise computations can allow the system to ignore distant airplanes when an imminent collision is detected and therefore an immediate notification to the pilot is absolutely necessary. Hence, imprecise computations can provide scheduling flexibility in real-time systems, by trading off precision for timeliness (Feng, 1996; Hull, 2000; Shih and Liu, 1992).

In distributed real-time systems, jobs usually consist of component tasks with precedence constraints among them, so that a task's output may be used as input by other tasks of the job. Thus, jobs are *directed acyclic graphs* (DAGs) – i.e. *task graphs* – that have an *end-to-end deadline*. That is, component tasks do not have any specific individual deadlines, but there is an end-to-end timing

* Corresponding author.
E-mail addresses: gstavrin@csd.auth.gr (G.L. Stavrinides), karatza@csd.auth.gr (H.D. Karatza).

0164-1212/$ - see front matter © 2009 Elsevier Inc. All rights reserved.
doi:10.1016/j.jss.2009.12.025

constraint over all the tasks of the job, that must be met. In order for a task to start execution, all of its predecessor tasks must have been completed. A task with no predecessors is called an *entry* task, whereas a task with no successors is called an *exit* task. The immediate predecessors of a task are called *parents* of the particular task, while the immediate successors of a task are called *children* of the particular task.

In the case of imprecise computations, each component task is assumed to be monotone and that it consists of a *mandatory part*, followed by an *optional part* (Liu et al., 1994; Chung et al., 1990; Han et al., 2003). In order for a task to produce an acceptable result, its mandatory part must be completed. The optional part refines the result produced by the mandatory part. That is, the precision of a task's result is further increased, if the task's optional part is allowed to be executed longer. *Input error* may affect the processing time of some tasks. Specifically, if a task's result is imprecise, then the execution time of its child tasks may increase, since more time may be required by the child tasks to produce an acceptable result when there is error in their input (Hull et al., 1997). In order for a job to be completed, all of its tasks must complete *at least* their mandatory part before the job's deadline.

Scheduling in distributed real-time systems has been studied by many authors (Karatza, 2007, 2008; Stavrinides and Karatza, 2008, 2009; Seljak, 1994; Sha et al., 2004). Among the real-time scheduling policies that have been proposed in the literature, the *Earliest Deadline First* (EDF) algorithm is the most commonly used (Liu and Layland, 1973). According to this technique, the deadlines of the jobs are used for their priority assignment. Task graph scheduling strategies are examined in Kwok and Ahmad (1999); Iverson and Ozguner (1999) and Chen and Maheswaran (2002), such as the *Highest Level First* (HLF) policy (Adam et al., 1974), which assigns priorities to the tasks according to their position in the graph. For the dynamic scheduling of multiple task graphs in multiprocessor real-time systems, Cheng et al. (1997) proposed a novel scheduling algorithm, called *Least Space–Time First* (LSTF), that takes into account both the precedence and the timing constraints among the tasks. Feng and Liu (1997) investigate the impact of input error on the static scheduling of a single linear chain of tasks with an end-to-end deadline on a single processor. On the other hand, Haweet et al. (2003) examine the static scheduling of a single task graph with an end-to-end deadline on a single processor, but without taking into account the effects of input error on the execution time of the component tasks.

In this paper, we examine the performance of the EDF, HLF and LSTF algorithms for the scheduling of multiple task graphs (i.e. composite jobs with a directed acyclic graph structure) with end-to-end deadlines in a homogeneous distributed real-time system. We also provide an alternative version for each of the three algorithms, which allows imprecise computations, taking into account the effects of input error on the processing time of the component tasks of a job. Our foremost goal is to guarantee that all jobs that arrive in the system will meet their deadlines, providing high quality (precise) results. We compare the performance of the scheduling policies by simulation, under various workloads, based on the Overall System Performance metric, which is explained in Section 5. To our knowledge, an imprecise computations approach for the dynamic scheduling of multiple task graphs with end-to-end deadlines and input error has never been discussed in the literature before.

The remainder of this paper is organized as follows: Section 2 gives a description of the system under study and the workload model, Section 3 describes the scheduling strategies, Section 4 explains how imprecise computations are incorporated into the scheduling process and Section 5 presents and analyzes the simulation results. Finally, Section 6 concludes this paper, providing suggestions for further research.

2. System and workload model

The target system is considered to be a homogeneous distributed real-time system, consisting of P processors, each serving its own queue (memory). Real-time jobs arrive in the system in a Poisson stream with rate λ. Each job J_k that arrives in the system, is a directed acyclic graph (DAG) of non-preemptible component tasks. Each vertex in the graph represents a task T_i of job J_k, whereas a directed edge E_{ij} between a task T_i and a task T_j represents a message that must be transmitted from task T_i to task T_j. A task can start execution only if it has received its required input data from all of its parent tasks. It is assumed that no additional data are required during a task's execution and that the output data of a task are available only after the task's completion.

Assigned to each edge (message) is a communication cost. The communication cost of a particular message between any pair of processors in the system is considered to be the same. Moreover, the communication cost between two precedence constrained tasks assigned to the same processor is considered to be negligible. It is assumed that the processors in the system are fully connected, that the communication is contention-free and that a processor can receive and send data to other processors simultaneously, even when it is executing a task. Thus, no attention is paid to the routing strategies used for communication. Hereafter, we will use the terms job, task graph and DAG interchangeably.

A job that arrives in the system may have one or more entry tasks and one or more exit tasks. Each component task T_i of a job J_k is characterized by the following parameters:

- Its service (execution) time S_i, which is exponential with mean $1/\mu$.
- Its priority value PV_i. The task with the smallest or largest priority value (depending on the scheduling algorithm which is employed) is considered to have the highest priority for scheduling.
- Its *level* L_i. The level of a task is the length of the longest path from the particular task to an exit task. The length of a path in the graph is the sum of the service times and the communication costs of all the tasks and edges on the path. The level of an exit task is equal to the task's service time.
- Its *rank* R_i, which denotes at which level of the graph J_k task T_i is located. For example, if T_i is an entry task, then its rank is $R_i = 1$, since it is located at the top level of the graph. The tasks that are located at the same graph level have the same rank and can be processed in parallel. A level in the graph should not be mistaken for the level L_i of a task T_i.
- The *set* $\mathscr{V}_i$ *of its parent tasks*.
- The *set* $\mathscr{Y}_i$ *of its child tasks*.

The communication cost C_{ij} of an edge E_{ij} between two tasks T_i and T_j in a job J_k is defined as follows:

$$C_{ij} = CCF_{ij} \cdot S_i \tag{1}$$

where CCF_{ij} is the *communication cost factor* of the edge E_{ij} and may be different for each edge in job J_k. S_i is the service time of the predecessor task T_i.

Consequently, a job J_k that arrives in the system is characterized by the following parameters:

- Its arrival time A_k.
- The number N_k of its component tasks. It is assumed that $1 \leqslant N_k \leqslant MAX$, where MAX is the maximum number of tasks in a graph.
- The *topological order list* $\mathscr{L}_k$ of its tasks. $\mathscr{L}_k$ is a list where all the tasks of the graph are placed in ascending order of their ranks and therefore, according to their topological order in the graph.

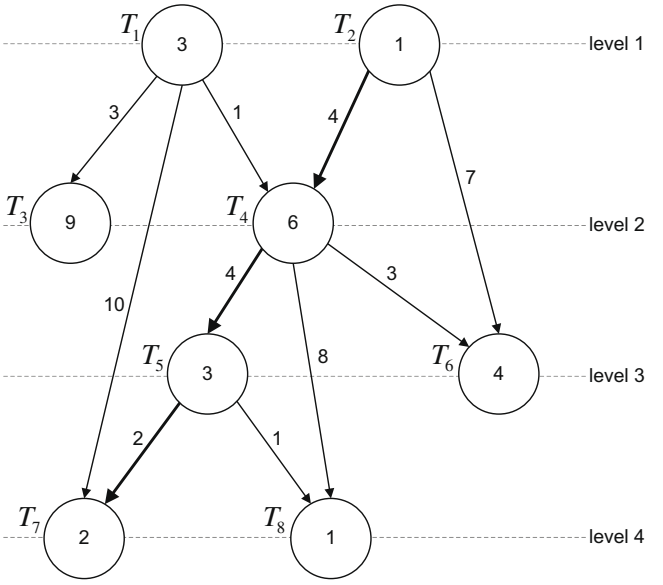


Fig. 1. An example task graph.

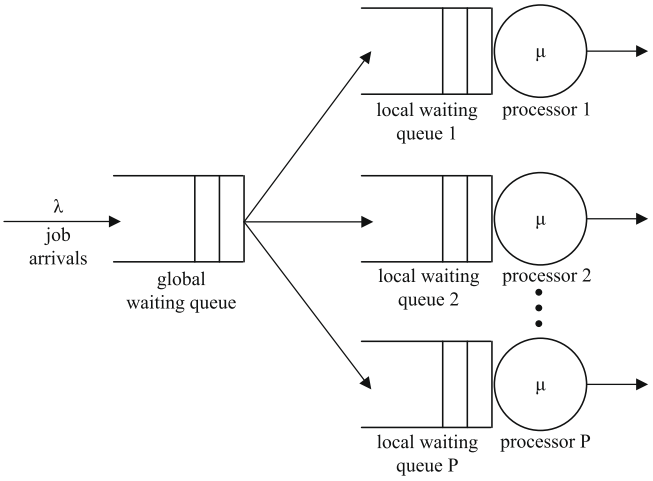


Fig. 2. The system model.

- The *set* $\mathscr{V}_k$ *of all parent tasks* in the graph.
- The *set* $\mathscr{Y}_k$ *of all child tasks* in the graph.
- The *set* $\mathscr{X}_k$ *of all exit tasks* in the graph.
- Its *critical path length* CPL_k, which is the length of the longest path from an entry task to an exit task in the graph.
- Its *height* H_k, which is equal to the number of levels in the graph.
- Its *absolute end-to-end deadline* D_k (measured from time zero).
- Its *relative end-to-end deadline* RD_k, which is the interval between J_k's arrival A_k and its absolute deadline D_k. That is,

$$RD_k = D_k - A_k. \tag{2}$$

- Its *cumulative service time* CST_k, which is equal to

$$CST_k = \sum_{Ti \in \mathscr{L}_k} S_i \tag{3}$$

where $\mathscr{L}_k$ is the list of all tasks of job J_k. S_i is the service time of a task T_i in $\mathscr{L}_k$.

An example task graph with 2 entry tasks and 4 exit tasks, is illustrated in Fig. 1. The number in each node denotes the task's service time, whereas the number on each edge denotes the communication cost of the edge. At each level of the graph, the tasks that can be processed in parallel are: on level 1 tasks T_1 and T_2 (their ranks are $R_1 = R_2 = 1$), on level 2 tasks T_3 and T_4 (their ranks are $R_3 = R_4 = 2$), on level 3 tasks T_5 and T_6 (their ranks are $R_5 = R_6 = 3$) and on level 4 tasks T_7 and T_8 (their ranks are $R_7 = R_8 = 4$). The cumulative service time of the task graph is $CST = 29$. The level of task T_2 is the highest in the graph and it is equal to $L_2 = 22$. The task graph's critical path length is equal to the highest task level in the graph, L_2. That is, $CPL = 22$. The edges on the critical path of the graph are shown with thick arrows.

In this paper, we assume that there is a centralized scheduler that has full knowledge of the system. When a task graph arrives in the system, all of its tasks first enter a *global waiting queue*. In this queue, the unscheduled tasks of all the arrived jobs wait until they get ready to be scheduled. A task is ready to be scheduled if it has no predecessors (i.e. if it is an entry task) or if all of its predecessors have finished execution. Thus, there are no data dependencies between tasks that are ready for scheduling. Once one or more tasks become ready to be scheduled, something which occurs after a job arrival or after a task completion, the scheduler is invoked. Each ready task is assigned a priority value according to the scheduling algorithm that is used. Subsequently, the ready tasks in the global waiting queue are scheduled starting from the one with the highest priority. The scheduler allocates a processor to each ready task and determines the position in the respective local waiting queue that each task will be placed, according to the task's priority. Once a task is placed in a local waiting queue, it may be required to wait to receive data from other processors. Otherwise, the task is ready for execution and therefore it waits for its turn to occupy the processor. All local waiting queues are arranged so that the task with the highest priority is placed at the head of the queue. Clearly, a local waiting queue may contain tasks of different jobs. We assume that task migrations are not permitted between the processors and that software or hardware failures do not occur during the execution of a task. The system model is illustrated in Fig. 2.

Obviously, in order for a task to be able to start execution, two conditions must be met:

- the task must be at the head of an idle processor's local waiting queue and
- all the required data from predecessor tasks must be available on the particular processor.

However, these conditions are not always satisfied at the same time.

A job is completed if all of its tasks are completed before the job's deadline. In the case where an uncompleted job's deadline is reached, the job is lost. Therefore, we discard all of its unscheduled tasks that are waiting in the global waiting queue, all of its scheduled tasks that are waiting in local waiting queues and its tasks that are in service. If some of the discarded tasks were receiving input data from other processors, those data transmissions are terminated.

3. Scheduling policies

3.1. Task selection

At each scheduling step, the task that is selected for scheduling among the ready tasks in the global waiting queue, is the one with the highest priority. If two or more tasks have the same priority, we choose randomly one of them. The priority of the tasks is calculated according to one of the following algorithms: Earliest Deadline First (EDF), Highest Level First (HLF) and Least Space–Time First (LSTF). In the case of EDF, the priority value PV_i

of a component task T_i is considered to be equal to the absolute deadline D_k of T_i's job J_k. That is,

$$PV_i = D_k \tag{4}$$

In this case, the task which has the smallest priority value (i.e. the earliest end-to-end deadline) is considered to have the highest priority for scheduling.

When HLF is employed for the scheduling of the tasks, the priority value PV_i of a task T_i is considered to be its level L_i:

$$PV_i = L_i \tag{5}$$

Thus, in this case the task with the highest priority is the one with the largest priority value (i.e. the task with the highest level).

On the other hand, if LSTF is used, then the priority value PV_i of a task T_i is considered to be its *space–time* ST_i. The space–time ST_i of a task T_i at time t, is defined as follows:

$$ST_i(t) = D_k - t - L_i \tag{6}$$

where D_k is the absolute deadline of T_i's job J_k and L_i is the level of T_i. Therefore, the priority value of a task T_i at time t is:

$$PV_i(t) = ST_i(t) \tag{7}$$

That is, in the case of LSTF, the task with the smallest priority value (i.e. the smallest space–time) at the particular time instant, has the highest priority. Since $PV_i(t)$ changes over time, it must be recalculated at each scheduling step.

3.2. Processor selection

Once a task is selected by the scheduler, it is allocated to the processor which can provide it with the earliest *estimated start time EST* (ties are broken randomly). The *EST* of a task T_i on a processor P_j is calculated as follows:

$$EST(T_i, P_j) = \max\{t_{data}(T_i, P_j), t_{idle}(T_i, P_j)\} \tag{8}$$

where $t_{data}(T_i, P_j)$ is the time interval after which all input data of task T_i will be available on processor P_j, whereas $t_{idle}(T_i, P_j)$ is the time interval after which task T_i will reach the head of P_j's local waiting queue and P_j will be idle and thus able to execute task T_i.

$t_{data}(T_i, P_j)$ is equal to the largest communication cost between a parent task of T_i and T_i. If a parent task of T_i was executed on processor P_j, we consider the respective communication cost to be equal to zero.

In order to calculate $t_{idle}(T_i, P_j)$, we find the *potential position* of task T_i in processor P_j's local waiting queue, according to T_i's priority (that is, the position at which T_i would be placed in P_j's queue if it was assigned to that particular processor). Consequently, if task T_i would be placed at the head of P_j's queue and P_j is busy executing another task, then $t_{idle}(T_i, P_j)$ is equal to the remaining execution time of the task in service. Otherwise, if P_j is idle, then $t_{idle}(T_i, P_j) = 0$. On the other hand, if T_i would be placed at any other position in P_j's queue, then $t_{idle}(T_i, P_j)$ is equal to:

$$t_{idle}(T_i, P_j) = EST(T_{previous}, P_j) + S_{previous} \tag{9}$$

where $T_{previous}$ is the task that would precede T_i in the particular queue and $S_{previous}$ is the service time of $T_{previous}$. It is assumed that the scheduling overhead is negligible.

4. Integration of imprecise computations

For each of the scheduling algorithms presented in the previous section, we provide an alternative version (*EDF_IC*, *HLF_IC* and *LSTF_IC*), which allows imprecise computations. To achieve this, we assume that every job that arrives in the system consists of component tasks that are *monotone*. Furthermore, it is assumed that each component task consists of a *mandatory part*, followed by an *optional part*.

In our approach, we consider that a component task is completed, if it has executed *at least* its mandatory part. That is, a task can either complete its optional part entirely, partially or it may skip its whole optional part, depending on the workload of the system. When a task completes only a part of its optional part (or if its entire optional part is omitted), then there is error in its results (*output error*) and therefore its results are *imprecise*. If the particular task is a parent of other tasks in the graph, then its output data will be used as input by its child tasks. Therefore, the child tasks of the particular task will have *input error*. We assume that each child task of a task T_i requires the same minimum result precision from T_i. That is, task T_i needs the same amount of time to produce a result that is acceptable by each of its child tasks.

If a task has input error, then there is an impact on its execution time. Specifically, we assume that the execution time of the mandatory part of the task is extended, since more time is required by the task to correct the error and produce an acceptable result. For instance, in weather forecasting systems, there is a satellite data collection task *A* and a data visualization task *B*, which produces the satellite images. Assume that the processing time of the mandatory part of task *B* is the time required by the task to process and visualize the data provided by task *A*, in order to produce an image of an acceptable standard. The processing time of the optional part of task *B* is the time required by the task to enhance the quality of the image produced by its mandatory part. If the data provided to task *B* by task *A* are imprecise (e.g. due to noise during the satellite transmission), then task *B* will require more time to process the data and produce an image of the acceptable standard. That is, the mandatory part of task *B* is extended due to input error. The optional part of task *B* remains the same.

In this paper, we assume that the optional part of a task is not affected by the task's input error and that a task can always eliminate its input error and produce precise results, by extending the processing time of its mandatory part. Moreover, it is assumed that the size of the data produced by a task – and thus the communication cost of the task's outgoing messages to its child tasks – is independent of the precision of the data. Every task in a task graph is assumed to have a non-zero mandatory part and a non-zero optional part.

Consequently, in the imprecise computations case, in addition to the parameters presented in Section 2, each task T_i of a job J_k is also characterized by the following parameters:

- Its *initial mandatory part* MP_i. It is assumed that $0 < MP_i < S_i$, where S_i is the initial service time of task T_i.
- Its *optional part* OP_i, which is defined as

$$OP_i = S_i - MP_i \tag{10}$$

where S_i is the initial service time and MP_i is the initial mandatory part of task T_i.

- Its *output error* OE_i, which is defined to be

$$OE_i = \frac{OP_i - p_i}{OP_i} \tag{11}$$

where p_i is the time spent on executing T_i's optional part OP_i. Thus, $OP_i - p_i$ is the amount of discarded work of task T_i. Since $0 \leqslant p_i \leqslant OP_i$, from (11), it follows that $0 \leqslant OE_i \leqslant 1$.

- Its *input error* IE_i, which is defined as

$$IE_i = \min\left\{1, \sum_{T_j \in \Upsilon_i} OE_j\right\} \tag{12}$$

where Υ_i is the set of all parent tasks of task T_i. OE_j is the output error of a parent task T_j in Υ_i. For example, the input error of the

tasks T_1, T_3 and T_7 in Fig. 1 is $IE_1 = 0$, $IE_3 = OE_1$ and $IE_7 = \min\{1, OE_1 + OE_5\}$, respectively.

- Its *mandatory part extension* MPE_i, which is equal to

$$MPE_i = IEF_i \cdot IE_i \tag{13}$$

where IEF_i is the *input error factor* of task T_i. IE_i is T_i's input error. The input error factor of task T_i is defined to be $0 < IEF_i < 2MP_i$, where MP_i is the initial mandatory part of task T_i.

- Its *extended mandatory part* MP'_i, which is defined as

$$MP'_i = MP_i + MPE_i \tag{14}$$

where MP_i is the initial mandatory part and MPE_i is the mandatory part extension of task T_i. In the case where task T_i has not any input error (i.e. $IE_i = 0$), its mandatory part remains the same. That is, $MP'_i = MP_i$.

- Its *extended service time* S'_i, which is defined to be

$$S'_i = MP'_i + OP_i = S_i + MPE_i \tag{15}$$

where MP'_i is the extended mandatory part, OP_i is the optional part, S_i is the initial service time and MPE_i is the mandatory part extension of task T_i. If the input error of task T_i is $IE_i = 0$, then $S'_i = S_i$.

- Its *extended level* L'_i, which is equal to

$$L'_i = L_i + MPE_i \tag{16}$$

where L_i is the initial level and MPE_i is the mandatory part extension of task T_i.

- Its *result precision threshold* RPT_i, which is the minimum result precision that is acceptable from task T_i. That is, RPT_i is the result precision provided by task T_i when only its extended mandatory part is completed. Each task of job J_k may have a different result precision threshold. It is assumed that $0 < RPT_i < 1$.
- Its *result precision* RP_i, which is defined as

$$RP_i = RPT_i + (1 - RPT_i)(1 - OE_i) \tag{17}$$

where RPT_i and OE_i are the result precision threshold and the output error of task T_i, respectively. The result precision of every completed task is $RPT \leqslant RP \leqslant 1$. When a task T_i completes both its extended mandatory part MP'_i and its optional part OP_i, it produces precise results (i.e. its output error is $OE_i = 0$) and therefore its result precision is $RP_i = 1$. On the other hand, if a task T_i completes only its extended mandatory part MP'_i, then it produces imprecise results of the minimum acceptable quality (i.e. its output error is $OE_i = 1$) and thus its result precision is $RP_i = RPT_i$. By extension, the result precision RP_k of a job J_k is defined as

$$RP_k = \frac{\sum_{T_i \in \mathscr{X}_k} RP_i}{|\mathscr{X}_k|} \tag{18}$$

where $\mathscr{X}_k$ is the set of all exit tasks of job J_k and RP_i is the result precision of an exit task T_i in $\mathscr{X}_k$. $|\mathscr{X}_k|$ is the cardinality of $\mathscr{X}_k$.

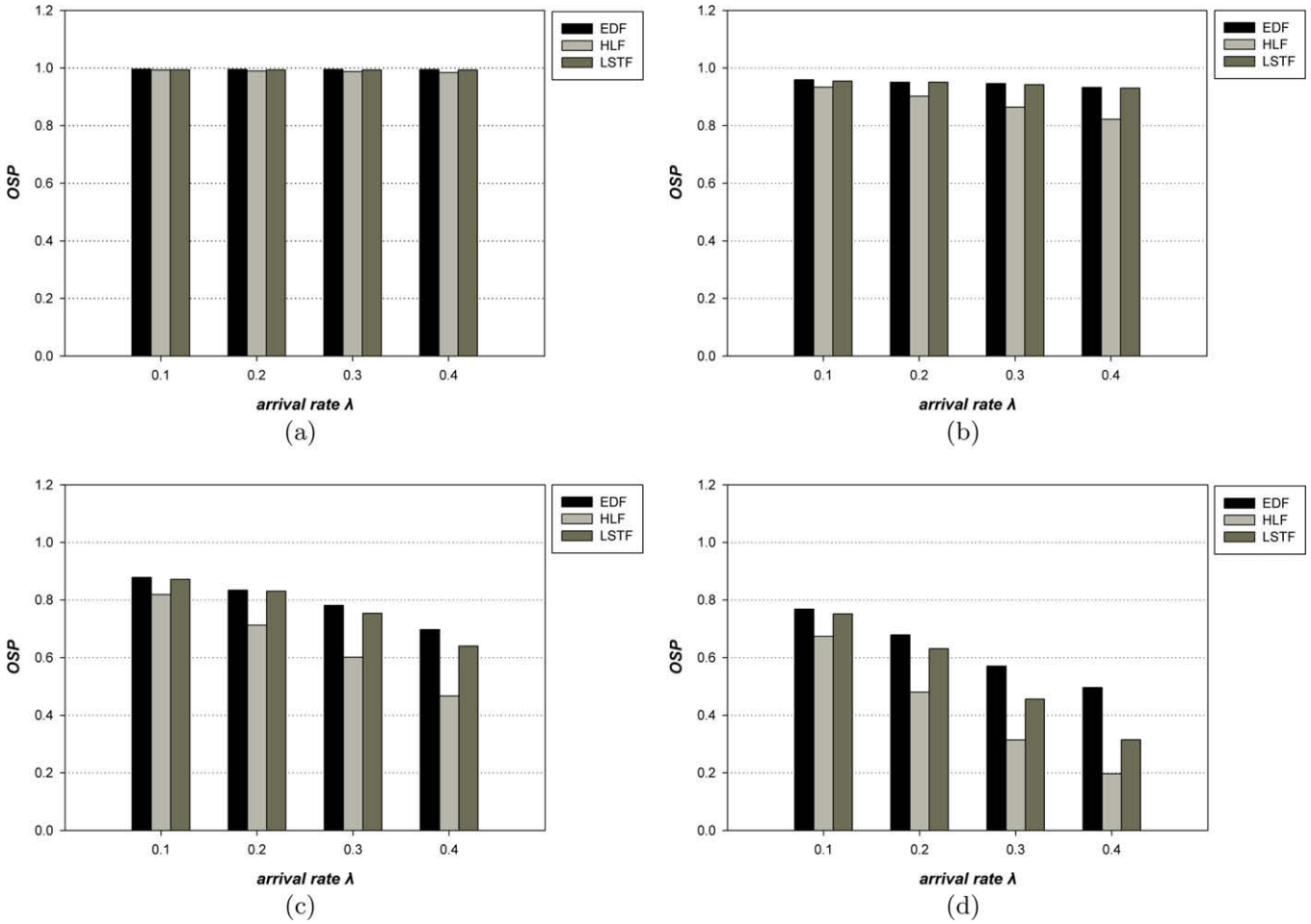


Fig. 3. *OSP* vs. arrival rate λ for EDF, HLF and LSTF, for: (a) $MAX = 16$, (b) $MAX = 32$, (c) $MAX = 48$ and (d) $MAX = 64$.

That is, $|\mathscr{X}_k|$ is the number of exit tasks of job J_k. Hence, the result precision of a job is equal to the average result precision of its exit tasks.

The initial mandatory part, the optional part, the input error factor and the result precision threshold of a task are calculated as soon as the task's job arrives in the system. The output error of a task is calculated when the task completes execution. If the particular task is an exit task, we also calculate its result precision. When a task becomes ready to be scheduled and it is a child task, we calculate its input error, its mandatory part extension, its extended mandatory part, its extended service time and its extended level. Subsequently, the task's priority value is calculated, according to the employed scheduling algorithm, as in the case where imprecise computations are not allowed.

In order to determine how much time to allocate for the execution of the optional parts of the tasks in a job, we developed a technique, called *Optional Parts Analysis* (OPA), which is used by the scheduler upon the arrival of a job. This technique is based on the following observations:

- When the optional part of a task T_i is partially or entirely omitted, then the processing time of T_i's child tasks is increased. Therefore, the cumulative service time of T_i's job J_k may increase, having as a result the delay of other jobs in the system, or even worse, this may lead job J_k to miss its deadline.

Table 1
Simulation input parameters.

Parameter description	Value
Number of processors of the distributed real-time system	$P = 32$
Service rate of the tasks of the jobs	$\mu = 1$
Number of component tasks in each job	$N \sim U[1, MAX]$
Communication cost factor of each edge in a task graph	$CCF \sim U[0.1, 10]$
Relative deadline of each job	$RD \sim U[CPL, 2CPL]$
Mandatory part of each task in a task graph	$MP \sim U(0, S)$
Input error factor of each task in a task graph	$IEF \sim U(0, 2MP)$
Result precision threshold of each task in a graph	$RPT \sim U(0, 1)$

- The quality of a job's results ultimately depends on the result precision of the job's exit tasks. Therefore, all exit tasks of a job should be allowed to complete their entire optional part.
- If the optional parts of all parent tasks (i.e. non-exit tasks) of a job J_k are omitted, then the input error of every child task T_i of job J_k will be equal to one. Therefore, from (13) it follows that the mandatory part extension of each child task T_i will be equal to T_i's input error factor. That is, $MPE_i = IEF_i$.

According to the OPA technique, if a job consists of only one task, then the task is scheduled so as to fully complete its optional part, unless the deadline of the job is reached. On the other hand, if there are more than one task in a job, then we calculate the sum of the optional parts of all parent tasks in the graph (which is the total time that can be saved by skipping the optional parts of all

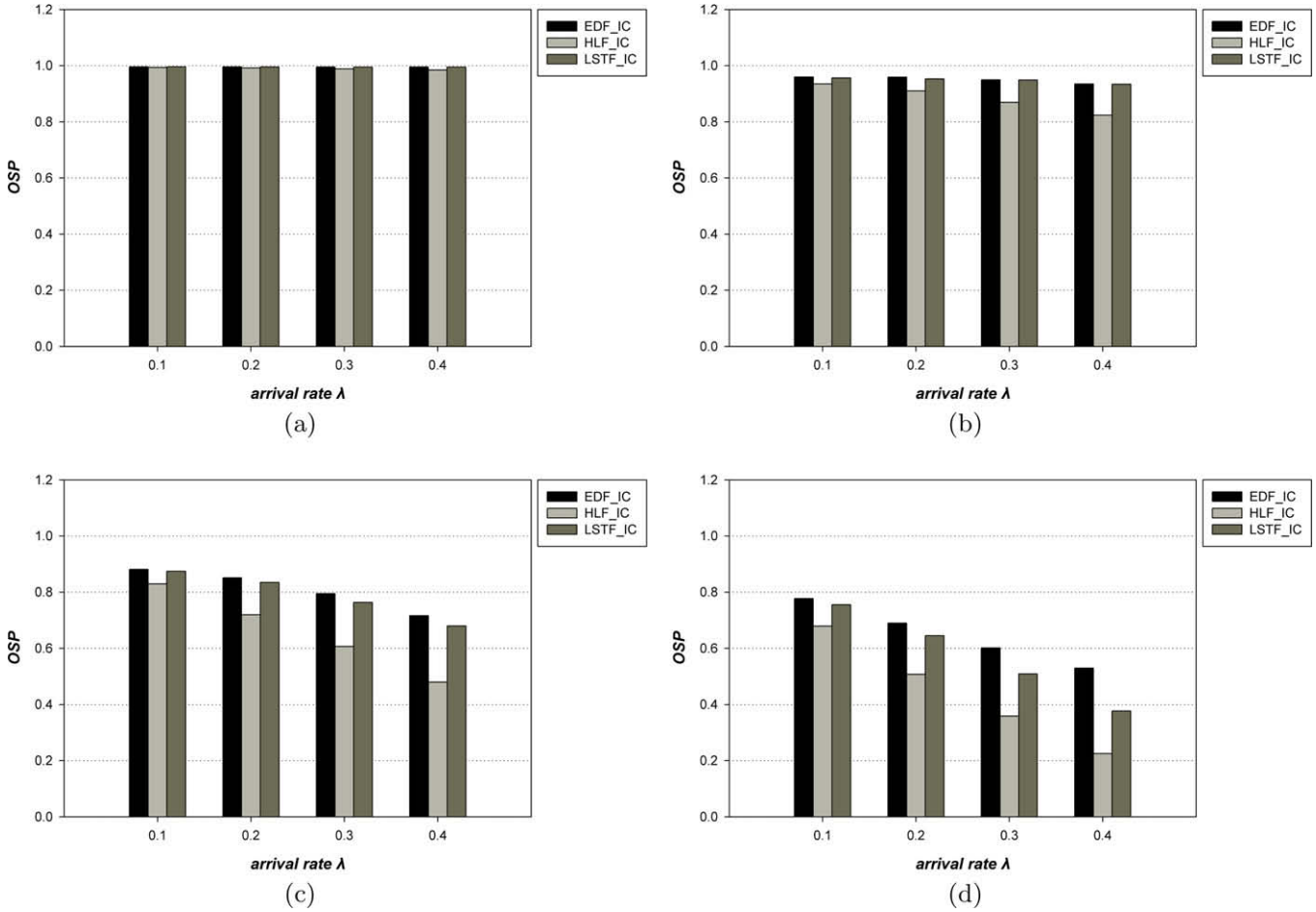


Fig. 4. *OSP* vs. arrival rate λ for EDF_IC, HLF_IC and LSTF_IC, for: (a) $MAX = 16$, (b) $MAX = 32$, (c) $MAX = 48$ and (d) $MAX = 64$.

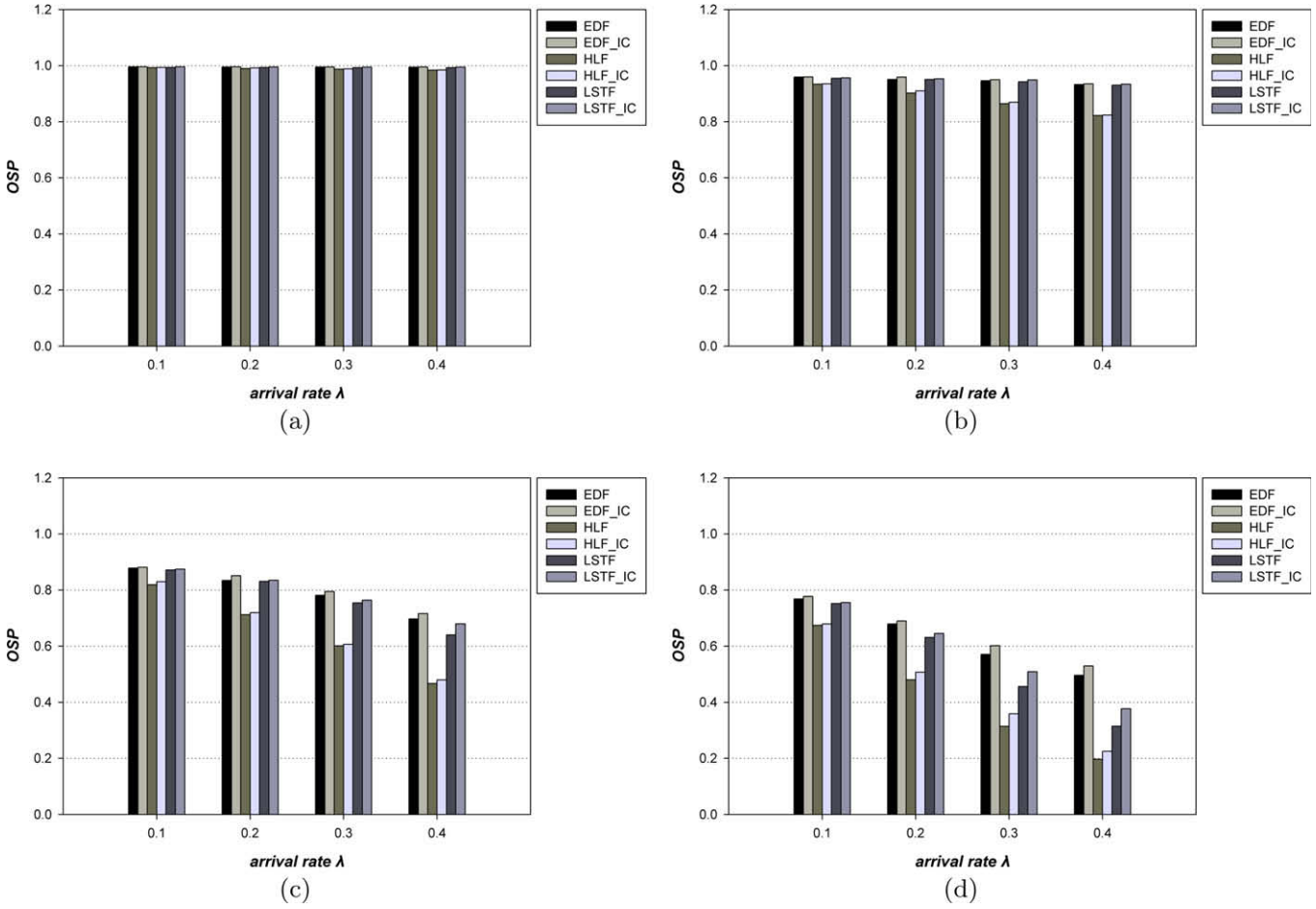


Fig. 5. *OSP* vs. arrival rate λ for EDF, EDF_IC, HLF, HLF_IC, LSTF and LSTF_IC, for: (a) $MAX = 16$, (b) $MAX = 32$, (c) $MAX = 48$ and (d) $MAX = 64$.

non-exit tasks) and if it is greater than or equal to the sum of the input error factors of all child tasks in the graph (which is the total extra execution time imposed by the output error of the partially completed parent tasks), then the optional parts of all parent tasks in the graph are entirely omitted, since it is certain that the cumulative service time of the job will be decreased.

On the contrary, if the sum of the optional parts of all parent tasks in the graph is less than the sum of the input error factors of all child tasks in the graph, then it is certain that by omitting the entire optional parts of all non-exit tasks will cause the cumulative service time of the job to increase. In this case, because input and output data dependencies can span an arbitrary number of tasks in the DAG, during the scheduling process, it is not computationally efficient to calculate how much time to allocate to the optional part of each parent task in the graph. Instead, the following heuristic is employed: each parent task in the graph is scheduled so as to complete its entire optional part. The processing of its optional part is interrupted, if there is a task ready for execution at the head of the particular queue. In this case, the task in service is aborted and the other task occupies the processor. Otherwise, the optional part of the parent task is fully completed (unless the deadline of the job is reached). In this way, we allocate to the optional parts of the parent tasks as much processing time as possible, without delaying other tasks in the system that are currently ready for execution. The exit tasks in the graph are always scheduled so as to complete their entire optional part, unless the deadline of the job is reached. The OPA technique is shown in Algorithm 1, where $\mathscr{V}_k$ is the set of all parent tasks, $\mathscr{Y}_k$ is the set of all child tasks and $\mathscr{X}_k$ is the set of all exit tasks of a job J_k.

Algorithm 1. Optional Parts Analysis (OPA)

Input: A job J_k.
Output: A decision on whether the optional part OP_i of each task T_i of job J_k will be omitted or not.

```
if job J_k consists of only one task T_i then
    schedule for execution the entire optional part OP_i of T_i
else
    if Σ_{T_i∈V_k} OP_i ⩾ Σ_{T_j∈Y_k} IEF_j then
        omit OP_i ∀T_i ∈ V_k
    else
        schedule for execution the entire OP_i ∀T_i ∈ V_k
        if there is a task ready for execution at the head of T_i's queue then
            interrupt the processing of T_i's optional part OP_i
        end if
    end if
    schedule for execution the entire OP_j ∀T_j ∈ X_k
end if
```

It is important to note that in the imprecise computations case, when the deadline of a job J_k is reached and some of its tasks are still in service, job J_k is not always considered lost. Contrarily, if J_k's tasks that are in service are the last exit tasks of J_k remaining to be completed and all of them have completed their mandatory part at the time of J_k's deadline, then the tasks, and thus job J_k, are considered completed.

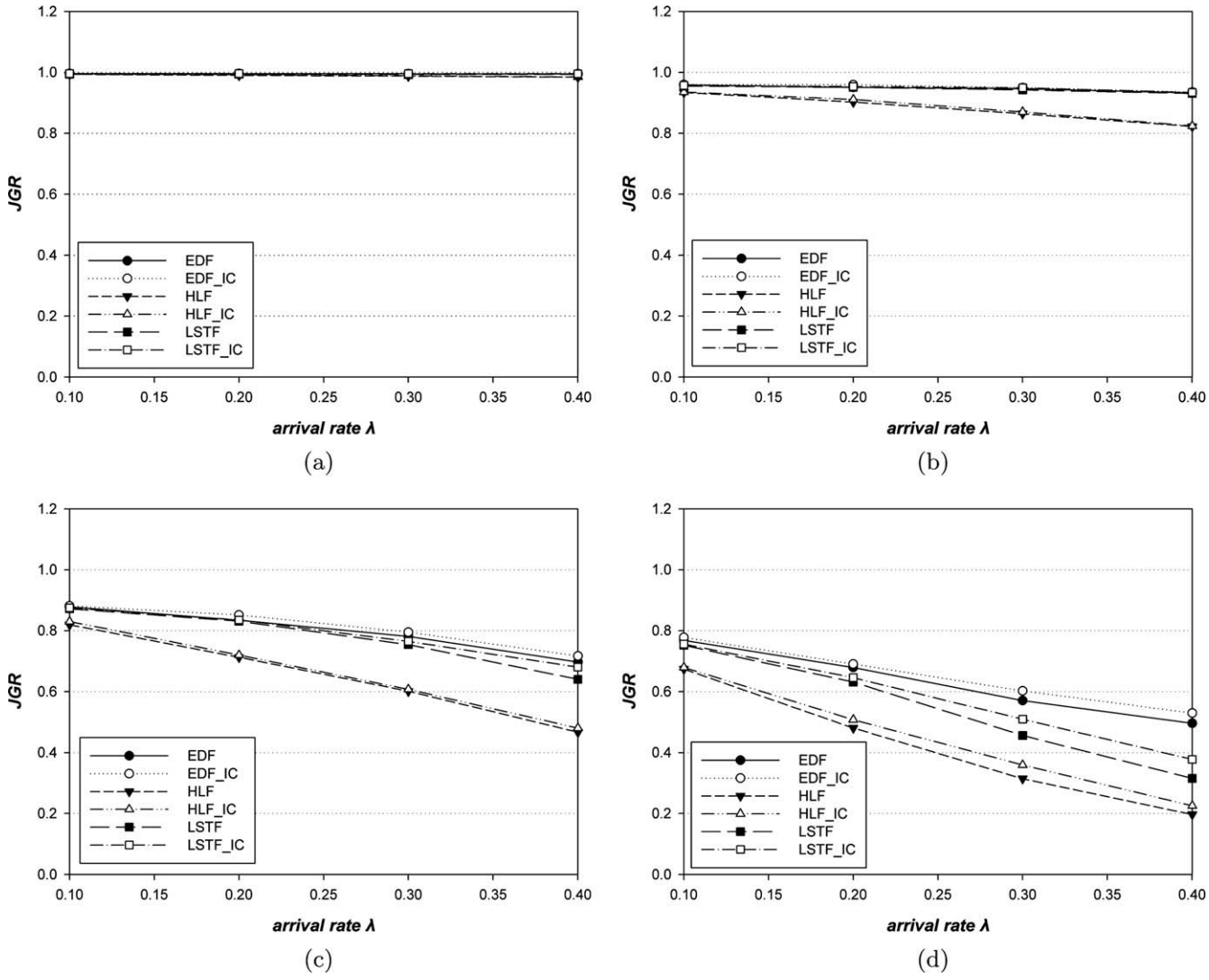


Fig. 6. *JGR* vs. arrival rate λ for EDF, EDF_IC, HLF, HLF_IC, LSTF and LSTF_IC, for: (a) $MAX = 16$, (b) $MAX = 32$, (c) $MAX = 48$ and (d) $MAX = 64$.

5. Performance evaluation

5.1. Performance parameters

For the performance evaluation of the EDF, HLF, LSTF, EDF_IC, HLF_IC and LSTF_IC scheduling policies in the framework described in the previous sections, we conducted a series of simulation runs using the independent replication method. Due to the complexity of the system and workload model under study, we implemented our own discrete-event simulation program in C++, tailored to the requirements of the specific case study. Since EDF, HLF and LSTF do not allow imprecise computations, whereas EDF_IC, HLF_IC and LSTF_IC do, our performance parameters need to take into account not only the number of jobs that meet their deadlines (that is, the number of *guaranteed* jobs), but also the precision of the results of the guaranteed jobs. Therefore, we employ a performance metric called *overall system performance* (*OSP*), which is defined as follows:

$$OSP = JGR \cdot ARP \tag{19}$$

where *JGR* is the *job guarantee ratio* and *ARP* is the *average result precision*. *JGR* is defined as

$$JGR = \frac{TNJG}{TNJA} \tag{20}$$

where *TNJG* is the *total number of jobs guaranteed* and *TNJA* is the *total number of job arrivals*. *ARP* is given by the expression

$$ARP = \frac{\sum_{J_k \in \mathscr{G}} RP_k}{|\mathscr{G}|} \tag{21}$$

where $\mathscr{G}$ is the set of all of the guaranteed jobs, RP_k is the result precision of a job J_k in $\mathscr{G}$ and $|\mathscr{G}|$ is the total number of jobs guaranteed. That is, $|\mathscr{G}| = TNJG$. It is important to note that in the case where imprecise computations are not allowed (EDF, HLF and LSTF), the result precision of a completed job J_k is always $RP_k = 1$. Therefore, the average result precision is always $ARP = 1$ and hence the overall system performance is $OSP = JGR$.

Clearly, from (20) and (21), (19) can be rewritten as:

$$OSP = \frac{\sum_{J_k \in \mathscr{G}} RP_k}{TNJA} \tag{22}$$

In the best case, all of the jobs that arrive in the system are guaranteed and are fully completed. That is, $TNJG = TNJA$, the result precision of all jobs is $RP = 1$ and thus $\sum_{J_k \in \mathscr{G}} RP_k = TNJA$. Therefore, from (22), $OSP = 1$. In the worst case, all of the jobs that arrive in the system are lost. That is, the set $\mathscr{G}$ of the guaranteed jobs is empty. Hence, $\sum_{J_k \in \mathscr{G}} RP_k = 0$ and therefore, from (22), $OSP = 0$. Thus, the overall system performance *OSP* is always in the range $[0,1]$.

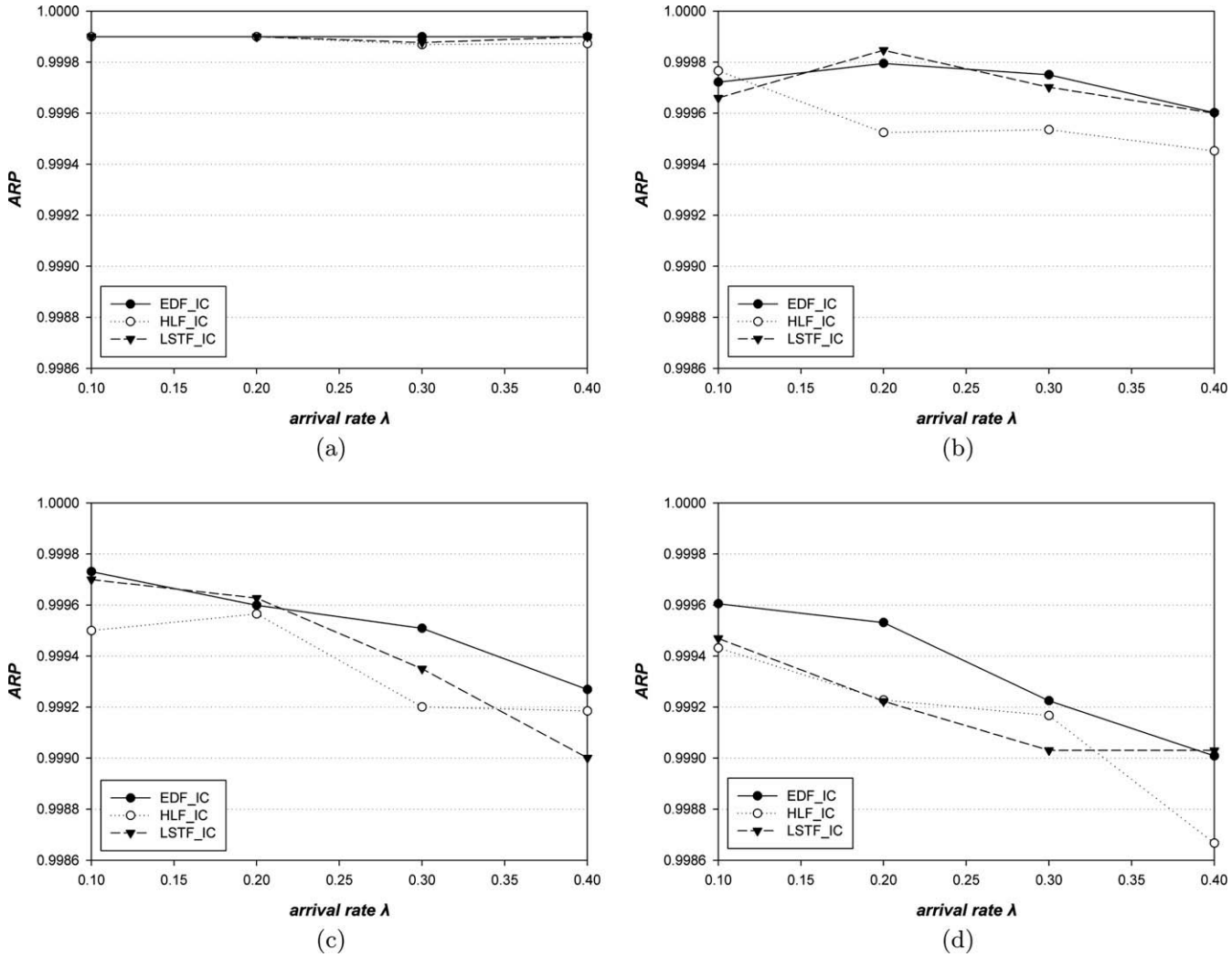


Fig. 7. *ARP* vs. arrival rate λ for EDF_IC, HLF_IC and LSTF_IC, for: (a) $MAX = 16$, (b) $MAX = 32$, (c) $MAX = 48$ and (d) $MAX = 64$.

5.2. Simulation results and discussion

5.2.1. Task graph generation

In our simulation experiments, the jobs (DAGs) were generated randomly. Each generated DAG is a *weakly connected* graph. That is, there is a path between any pair of tasks in the graph, without taking into account the direction of the edges. In the case where a graph has more than one task, there is at least one entry and one exit task in the graph. In order to generate a DAG, we first determine the number of tasks N in the graph. N is uniformly distributed in the range $[1, MAX]$, where MAX is the maximum number of tasks in a graph. Subsequently, if $N > 1$, we determine the number of entry tasks N_{entry} in the graph. N_{entry} is uniformly distributed in the range $[1, N-1]$. We then create the first entry task T_l and we set its rank R_l to be equal to one. Task T_l is then placed into the topological order list $\mathscr{L}$.

Subsequently, the $N - N_{entry}$ non-entry tasks (which may be exit tasks – this is determined when the construction of the task graph is finally completed) of the graph are created, one by one. At each step, we create a task T_j and we determine the number $|\mathscr{V}_j|$ of its parent tasks. The number of T_j's parent tasks $|\mathscr{V}_j|$ is uniformly distributed in the range $[1, |\mathscr{L}|]$, where $|\mathscr{L}|$ is the number of the tasks that have already been created and placed in the topological order list $\mathscr{L}$. After determining $|\mathscr{V}_j|$, we randomly choose a task T_i from list $\mathscr{L}$ and we set it to be a parent of task T_j, by creating an edge E_{ij} which connects the two tasks T_i and T_j. The above procedure is repeated, until $|\mathscr{V}_j|$ parent tasks are connected to task T_j. The rank R_j of task T_j is equal to the highest rank of its parent tasks plus one. One is added, because task T_j must be at a lower level in the graph than its parent tasks. Subsequently, task T_j is placed in the topological order list $\mathscr{L}$ according to its rank. In the case of a tie, the most recently created task is placed after the tasks that are already in $\mathscr{L}$.

After creating $N - N_{entry}$ non-entry tasks, we continue constructing the DAG, by creating the rest $N_{entry} - 1$ entry tasks, one by one. At each step, we create a task T_m and we determine the number $|\mathscr{Y}_m|$ of its child tasks. The number of T_m's child tasks $|\mathscr{Y}_m|$ is uniformly distributed in the range $[1, N - N_{entry}]$. Afterwards, we randomly choose a non-entry task T_n from the topological order list $\mathscr{L}$ and we set it to be a child of task T_m, by creating an edge E_{mn} which connects the two tasks T_m and T_n. The above procedure is repeated, until $|\mathscr{Y}_m|$ child tasks are connected to task T_m. Since task T_m is an entry task, its rank is $R_m = 1$. Subsequently, task T_m is placed in the topological order list $\mathscr{L}$ according to its rank. It is important to note that in the above procedure, when connecting a child task to an entry task, the rank of the child task and therefore its order in list $\mathscr{L}$, is not affected. The pseudocode for our random DAG generator is shown in Algorithm 2. A task graph generated by this method has height $H = 1$ if it consists of only one task (i.e. $N = 1$). Otherwise, if the task graph consists of more than one task (i.e. $N > 1$), then its height is $2 \leqslant H \leqslant N$.

Algorithm 2. Random DAG Generator

Input: The maximum number *MAX* of tasks in the DAG.
Output: A DAG in the form of a topological order list $\mathscr{L}$ of its tasks.

1: determine the number of tasks in the graph $N \sim U[1, MAX]$
2: **if** $N > 1$ **then**
3: determine the number of entry tasks in the graph $N_{entry} \sim U[1, N-1]$
4: create the first entry task T_l
5: set $R_l = 1$
6: place task T_l in the topological order list $\mathscr{L}$
7: **repeat**
8: create a task T_j
9: determine the number of T_j's parent tasks $|\mathscr{V}_j| \sim U[1, |\mathscr{L}|]$
10: **repeat**
11: choose randomly a task T_i in $\mathscr{L}$
12: create edge E_{ij} that connects tasks T_i and T_j
13: **until** $|\mathscr{V}_j|$ parent tasks are connected to task T_j
14: set $R_j = \max_{T_i \in \mathscr{V}_j}\{R_i\} + 1$
15: place task T_j in the topological order list $\mathscr{L}$ according to its rank R_j
16: **until** $N - N_{entry}$ non-entry tasks are created
17: **repeat**
18: create a task T_m
19: determine the number of T_m's child tasks $|\mathscr{Y}_m| \sim U[1, N - N_{entry}]$
20: **repeat**
21: choose randomly a non-entry task T_n in $\mathscr{L}$
22: create edge E_{mn} that connects tasks T_m and T_n
23: **until** $|\mathscr{Y}_m|$ child tasks are connected to task T_m
24: set $R_m = 1$
25: place task T_m in the topological order list $\mathscr{L}$ according to its rank R_m
26: **until** $N_{entry} - 1$ entry tasks are created
27: **else**
28: create the only task T in the graph
29: **end if**

5.2.2. Simulation input parameters

For our experiments, we used the simulation input parameters shown in Table 1. Note that the parameters *MP*, *IEF* and *RPT* are used only in the imprecise computations case. We evaluated the performance of each algorithm based on the *OSP* metric, with respect to the arrival rate λ of the jobs, for $\lambda = 0.1, 0.2, 0.3, 0.4$. For each arrival rate λ, we considered that the maximum number of tasks in each job is $MAX = 16, 32, 48, 64$. We ran 30 replications of the simulation for each set of input parameters, with different seeds of random numbers and for 10^6 completed (that is, guaranteed) jobs in each replication. For every mean value, a 95% confidence interval was evaluated. The half-widths of all confidence intervals are less than 5% of their respective mean values. We conducted two sets of experiments: one set for the case where imprecise computations are not allowed (i.e. for the original versions of EDF, HLF and LSTF) and another set for the imprecise computations case (i.e. for the alternative versions of the algorithms, EDF_IC, HLF_IC and LSTF_IC). The simulation results are analyzed below.

5.2.3. Simulation results for EDF, HLF and LSTF

We first compared the performance of EDF, HLF and LSTF algorithms, with respect to the arrival rate λ, for $MAX = 16, 32, 48, 64$. This comparison is depicted in Fig. 3. Even though we expected that LSTF would exhibit the best performance, since it assigns priorities to the tasks taking into account both the deadline of their job as well as the level of each task in the graph, our simulation results show that EDF is the best performing policy, despite the fact that it considers only the deadlines of the jobs for the task priority assignment. On the other hand, HLF exhibits the worst performance, as expected, since it takes into account only the level of each component task in a graph, ignoring the timing constraints.

Specifically, when the maximum number of tasks in a graph is $MAX = 16$, the overall system performance is $0.9945 \leqslant OSP \leqslant 0.9959$ in the case of EDF, in the case of HLF is $0.9842 \leqslant OSP \leqslant 0.9932$ and in the case of LSTF is $0.9928 \leqslant OSP \leqslant 0.9936$. On the other hand, when $MAX = 64$, the overall system performance is $0.4960 \leqslant OSP \leqslant 0.7683$ in the case of EDF, in the case of HLF is $0.1965 \leqslant OSP \leqslant 0.6740$ and in the case of LSTF is $0.3151 \leqslant OSP \leqslant 0.7520$. As the job arrival rate λ and the maximum number *MAX* of tasks in a job increase, the overall system performance *OSP* decreases, for all policies. This is due to the heavier workload, which causes higher waiting times for the tasks in the queues. Therefore, more jobs miss their deadline.

However, between light workload ($\lambda = 0.1$ and $MAX = 16$) and heavy workload ($\lambda = 0.4$ and $MAX = 64$), the performance of EDF decreases by 50.20%, whereas the performance of HLF and LSTF decreases by 80.22% and by 68.29%, respectively. That is, EDF, compared to the other two algorithms, exhibits a more stable performance with respect to the system workload. Since in this case imprecise computations are not allowed, the average result precision is $ARP = 1$ for all strategies. Hence, the performance of the three policies depends only on the *JGR* parameter. That is, *OSP* depends only on the probability of not missing a job's deadline.

5.2.4. Simulation results for EDF_IC, HLF_IC and LSTF_IC

Subsequently, we examined the performance of the alternative versions of the algorithms, that allow imprecise computations. The comparison of EDF_IC, HLF_IC and LSTF_IC is shown in Fig. 4. As in the case of the original versions of the algorithms, in this case EDF_IC outperforms the other two algorithms, while HLF_IC exhibits the poorest overall system performance.

Specifically, when the maximum number of tasks in a graph is $MAX = 16$, the overall system performance is $0.9951 \leqslant OSP \leqslant 0.9961$ in the case of EDF_IC, $0.9846 \leqslant OSP \leqslant 0.9936$ in the case of HLF_IC and $0.9948 \leqslant OSP \leqslant 0.9958$ in the case of LSTF_IC. On the contrary, when $MAX = 64$, the overall system performance is $0.5291 \leqslant OSP \leqslant 0.7775$ in the case of EDF_IC, in the case of HLF_IC is $0.2251 \leqslant OSP \leqslant 0.6788$ and in the case of LSTF_IC is $0.3767 \leqslant OSP \leqslant 0.7554$. Between light workload ($\lambda = 0.1$ and $MAX = 16$) and heavy workload ($\lambda = 0.4$ and $MAX = 64$), the performance of EDF_IC decreases by 46.88%, whereas the performance of HLF_IC and LSTF_IC is decreased by 77.35% and by 58.38%, respectively. That is, EDF_IC, compared to HLF_IC and LSTF_IC, as its respective counterpart, exhibits a more stable performance with respect to the workload of the system. Moreover, the decrease in the performance of EDF_IC, HLF_IC and LSTF_IC when the system workload increases, is less than in the case of EDF, HLF and LSTF, respectively.

Fig. 5 depicts the comparison between the original and the alternative versions of EDF, HLF and LSTF. In the case of EDF_IC, the average improvement in the overall system performance when $MAX = 64$ is 3.74%, compared to EDF. In the case of HLF_IC the average improvement is 8.76%, compared to HLF, whereas in the case of LSTF_IC the average improvement is 8.44%, compared to LSTF. Despite the fact that HLF_IC exhibits the greatest improvement and EDF_IC the smallest, EDF_IC is still the best performing policy.

In the imprecise computations case, the performance of the algorithms depends on both of the parameters *JGR* (Fig. 6) and *ARP* (Fig. 7). In terms of the average result precision *ARP* parameter, all algorithms exhibit similar performance. Moreover, *ARP* is in the

range $[0.9987, 0.9999]$ for all policies. Therefore, the loss of result precision is not significant in the imprecise computations case. On the other hand, in terms of the job guarantee ratio *JGR* parameter, the improvement is remarkable, compared to the case where imprecise computations are not allowed. Specifically, when the maximum number of tasks in a graph is $MAX = 64$, the average improvement is 3.81% in the case of EDF_IC, compared to EDF. In the case of HLF_IC the average improvement is 8.87%, compared to HLF, whereas in the case of LSTF_IC the average improvement is 8.53%, compared to LSTF. Since both *JGR* and *ARP*, according to (19), have the same weight in the overall system performance, the great improvement in the job guarantee ratio dominates the negligible loss of average result precision. Therefore, EDF_IC, HLF_IC and LSTF_IC perform better than EDF, HLF and LSTF policies, respectively. EDF_IC exhibits the best performance among the investigated strategies.

Despite the fact that in our imprecise computations approach we take into account input error that may cause larger execution times, the alternative versions of the algorithms outperform their respective original versions. That is, even though the mandatory parts of some tasks may be extended due to the output error of their parent tasks, the scheduling flexibility provided by the trade-off between result precision and timeliness, ultimately gives a higher overall system performance.

6. Conclusions and future work

In this paper, we evaluated by simulation the performance of the EDF, HLF and LSTF policies for the scheduling of multiple task graphs with end-to-end deadlines in a homogeneous distributed real-time system. For each algorithm, we provided an alternative version (EDF_IC, HLF_IC and LSTF_IC), by incorporating imprecise computations into our model, taking into account the effects of input error on the processing time of the component tasks of a job. The performance of the scheduling strategies, in terms of the overall system performance *OSP* metric, was examined by simulation, under various workloads. Our simulation results show that in this case study, the alternative versions of the algorithms perform better than their original counterparts. Moreover, the EDF_IC policy exhibits promising performance and outperforms all the other policies.

However, in this paper we did not take into account the case where the system is heterogeneous. Furthermore, our scheduling policies could be further enhanced, by exploiting schedule holes that may appear due to the data dependencies of the tasks. Therefore, further research is needed in order to address these issues.

References

Adam, T.L., Chandy, K.M., Dickson, J.R., 1974. A comparison of list schedules for parallel processing systems. Communications of the ACM 17 (12), 685–690.

Buttazzo, G.C., 2004. Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications, second ed. Springer-Verlag, New York.

Chen, H., Maheswaran, M., 2002. Distributed dynamic scheduling of composite tasks on grid computing systems. In: Proceedings of the 2002 International Parallel and Distributed Processing Symposium (IPDPS 2002), Fort Lauderdale, FL, April 2002, pp. 88–97.

Cheng, B.C., Stoyenko, A.D., Marlowe, T.J., Baruah, S.K., 1997. LSTF: a new scheduling policy for complex real-time tasks in multiple processor systems. Automatica 33 (5), 921–926.

Chung, J.Y., Liu, J.W.S., Lin, K.J., 1990. Scheduling periodic jobs that allow imprecise results. IEEE Transactions on Computers 39 (9), 1156–1174.

Feng, W.C., 1996. Applications and extensions of the imprecise computation model. Ph.D. Thesis, University of Illinois at Urbana–Champaign.

Feng, W.C., Liu, J.W.S., 1997. Algorithms for scheduling real-time tasks with input error and end-to-end deadlines. IEEE Transactions on Software Engineering 23 (2), 93–106.

Han, C.C., Shin, K.G., Wu, J., 2003. A fault-tolerant scheduling algorithm for real-time periodic tasks with possible software faults. IEEE Transactions on Computers 52 (3), 362–372.

Haweet, W.A.E., Meligy, H.H.E., Salam, I.A.E., 2003. Imprecise computation technique to schedule and/or tasks with global end-to-end deadline in distributed real-time systems. In: Proceedings of the SICE 2003 Annual Conference, Fukui, Japan, August 2003, pp. 663–668.

Hull, D.L., 2000. An environment for imprecise computations. Ph.D. Thesis, University of Illinois at Urbana–Champaign.

Hull, D., Shankar, A., Nahrstedt, K., Liu, J.W.S., 1997. An end-to-end QOS model and management architecture. In: Proceedings of the IEEE Workshop on Middleware for Distributed Real-time Systems and Services, San Francisco, CA, December 1997, pp. 82–89.

Iverson, M.A., Ozguner, F., 1999. Hierarchical, competitive scheduling of multiple dags in a dynamic heterogeneous environment. Distributed Systems Engineering 6 (3), 112–120.

Karatza, H.D., 2007. Performance of gang scheduling policies in the presence of critical sporadic jobs in distributed systems. In: Proceedings of the 2007 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'07), San Diego, CA, July 2007, pp. 547–554.

Karatza, H.D., 2008. The impact of critical sporadic jobs on gang scheduling performance in distributed systems. Simulation: Transactions of the Society for Modeling and Simulation International 84 (2–3), 89–102.

Kwok, Y.K., Ahmad, I., 1999. Static scheduling algorithms for allocating directed task graphs to multiprocessors. ACM Computing Surveys 31 (4), 406–471.

Lin, K.J., Natarajan, S., Liu, J.W.S., 1987. Imprecise results: utilizing partial computations in real-time systems. In: Proceedings of the Eighth IEEE Real-Time Systems Symposium (RTSS'87), San Jose, CA, December 1987, pp. 210–217.

Liu, C.L., Layland, J.W., 1973. Scheduling algorithms for multiprogramming in a hard real-time environment. Journal of the ACM 20 (1), 46–61.

Liu, J.W.S., Lin, K.J., Shih, W.K., Yu, A.C., Chung, J.Y., Zhao, W., 1991. Algorithms for scheduling imprecise computations. Computer, IEEE 24 (5), 58–68.

Liu, J.W.S., Shih, W.K., Lin, K.J., Bettati, R., Chung, J.Y., 1994. Imprecise computations. Proceedings of the IEEE 82 (1), 83–94.

Seljak, B.K., 1994. Task scheduling policies for real-time systems. Microprocessors and Microsystems 18 (9), 501–511.

Sha, L., Abdelzaher, T., Arzen, K.E., Cervin, A., Baker, T., Burns, A., Buttazzo, G., Caccamo, M., Lehoczky, J., Mok, A.K., 2004. Real time scheduling theory: a historical perspective. Real-Time Systems 28 (2–3), 101–155.

Shih, W.K., Liu, J.W.S., 1992. On-line scheduling of imprecise computations to minimize error. In: Proceedings of the 13th IEEE Real-Time Systems Symposium (RTSS'92), Phoenix, AZ, December 1992, pp. 280–289.

Stavrinides, G.L., Karatza, H.D., 2008. Performance evaluation of gang scheduling in distributed real-time systems with possible software faults. In: Proceedings of the 2008 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS'08), Edinburgh, UK, June 2008, pp. 1–7.

Stavrinides, G.L., Karatza, H.D., 2009. Fault-tolerant gang scheduling in distributed real-time systems utilizing imprecise computations. Simulation: Transactions of the Society for Modeling and Simulation International 85 (8), 525–536.

Georgios L. Stavrinides is a Ph.D. student in the Department of Informatics, Aristotle University of Thessaloniki, Greece. He received the M.Sc. degree in Advanced Computing from Imperial College, London, UK in 2007. His research interests include performance evaluation of distributed real-time systems, simulation and scheduling algorithms.

Helen D. Karatza is a Professor in the Department of Informatics, Aristotle University of Thessaloniki, Greece. Her research interests include performance evaluation of parallel and distributed systems, cluster and grid computing, scheduling and simulation.